

Оптимизации выполнения: ускорение конвейера

Предсказание, спекуляция, переупорядочивание и
многопоточность

Содержание

- 1. Почему обычный pipeline перестаёт ускоряться
- 2. Предсказание условных переходов
- 3. Спекулятивное выполнение
- 4. Внеочередное выполнение команд
- 5. Многопоточность
- 6. Сравнение подходов
- 7. Почему следующим bottleneck становится память

1. Почему обычный конвейер может не ускорять процессор

1. Почему конвейер ускоряет процессор

Несколько инструкций находятся на разных этапах одновременно

Такт:	1	2	3	4	5		
I1	IF	ID	EX	MEM	WB		
I2		IF	ID	EX	MEM	WB	
I3			IF	ID	EX	MEM	WB

- ускоряется поток инструкций
- в идеале ≈ 1 инструкция за такт
- на практике конвейер простаивает

1. Причины замедления конвеера

Причина	Что происходит
Branch	неизвестен следующий <code>PC</code>
Data dependency	инструкция ждёт результат предыдущей инструкции
Memory latency	память отвечает медленнее, чем хотелось бы

Конвеер хорошо работает, пока идет работа на всех стадиях
Проблема: `stalls` и `bubbles` оставляют стадии пустыми

1. Почему нельзя бесконечно углублять pipeline

Более глубокий pipeline:

- + выше частота
- + меньше логики на стадию

Но:

- выше цена stall
- выше цена branch misprediction
- больше стадий надо очищать при ошибке

Вывод: одного только pipeline недостаточно

Нужны специальные техники ускорения

1. Где заканчивается ISA и начинается микроархитектура

ISA	Микроархитектура
Какие инструкции существуют	Как именно ядро их исполняет
<code>add</code> , <code>lw</code> , <code>beq</code> , <code>jal</code>	pipeline, prediction, OoO, multithreading
Поведение программы	Способ реализации

Одна и та же RV32I-программа может выполняться:

- на простом in-order ядре
- на pipeline-ядре
- на более сложном OoO-ядре

2. Предсказание условных переходов

2. Почему branch мешает pipeline

Обычная инструкция:

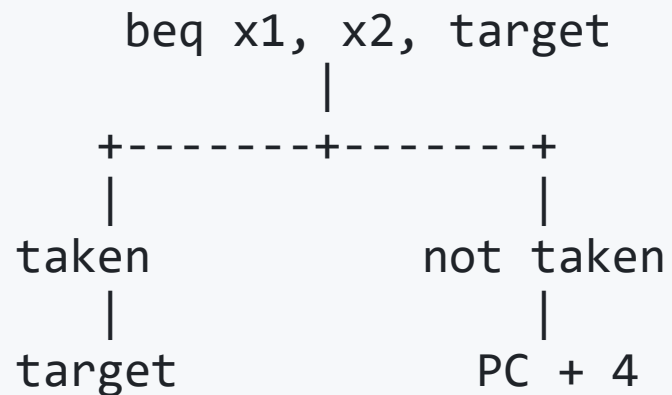
$PC_{next} = PC + 4$

Условный переход:

`beq x1, x2, target`

нужно узнать:

- 1) истинно ли условие?
- 2) куда идти дальше?



2. Самый простой вариант — ждать

Встретили branch



Остановили fetch



Дождались результата сравнения



Только потом выбрали следующий PC

- просто реализовать
- но pipeline простаивает
- особенно плохо для циклов и частых `if`

2. Идея branch prediction

Branch -> Predictor -> Predicted next PC -> Fetch

- не ждать результат branch
- угадать следующий PC
- продолжить fetch сразу

Если угадали: почти нет потери

Если ошиблись: придётся откатиться

2. Статическое и динамическое предсказание

Подход	Идея	Плюс	Минус
Статическое	всегда taken / not taken	очень просто	хуже точность
Динамическое	использовать историю	лучше точность	нужна доп. логика

Ключевая мысль:

поведение branch в программе часто повторяется

2. 1-bit predictor

Для branch храним 1 бит:

1 -> в прошлый раз был taken

0 -> в прошлый раз был not taken

Пример цикла:

taken	taken	taken	taken	not taken
✓	✓	✓	✓	✗

Проблема:

- на границе цикла часто бывают лишние ошибки

2. 2-bit predictor

Состояния:

[Strongly NT] $\leftrightarrow$ [Weakly NT] $\leftrightarrow$ [Weakly T] $\leftrightarrow$ [Strongly T]

Состояние	Предсказание
Strongly NT	not taken
Weakly NT	not taken
Weakly T	taken
Strongly T	taken

Плюс: одно случайное отклонение
не переворачивает предсказание сразу

2. Что обычно ещё нужно предсказывать

Недостаточно только ответа `taken / not taken`

Нужно ещё быстро узнать:

- `target address`
- то есть адрес, куда идти при `taken`

Fetch хочет как можно раньше получить:
следующий PC

Именно поэтому рядом с predictor часто есть:

- история branch
- буфер адресов переходов

3. Спекулятивное выполнение

3. Предсказание полезно только вместе со спекуляцией

Если процессор просто угадал branch и нет дальнейших переходов — выигрыша производительности нет.

Нужно идти вперёд:

```
Predict branch
↓
Fetch по предсказанному пути
↓
Decode следующих инструкций
↓
Выполнять их заранее
```

Это и есть спекулятивное выполнение

3. Что значит “спекулятивно”

- ядро действует как будто branch уже выполнен
- инструкции после branch идут дальше по конвейеру
- но их результат ещё не окончательный

```
Branch -> I2 -> I3 -> I4  
  \ speculative path /
```

3. Что происходит при ложном предсказании

Branch predicted wrong



Инструкции с неверного пути надо отбросить



Восстановить правильный PC



Начать fetch заново

[Branch] -> [I2 wrong] -> [I3 wrong] -> [I4 wrong]
flush flush flush

- pipeline очищается
- теряются такты
- чем глубже pipeline, тем дороже ошибка

3. Executed vs Committed

Состояние	Что это значит
Executed	результат вычислен внутри ядра
Committed	результат зафиксирован

Результат выполнения нескольких инструкций важно не только вычислить, но и зафиксировать

4. Внеочередное выполнение команд

4. Почему in-order execution тормозит

```
lw    t0, 0(a0)
add    t1, t0, t2
mul    t3, t4, t5
sub    t6, t7, t8
```

- `add` ЗАВИСИТ ОТ `lw`
- `mul` и `sub` ОТ `lw` НЕ ЗАВИСЯТ

Проблема in-order ядра:

если `lw` задержался, может простаивать и всё остальное

4. Идея out-of-order execution

```
Program order:  I1 -> I2 -> I3 -> I4  
Execute order:  I1 -> I3 -> I4 -> I2  
Commit order:   I1 -> I2 -> I3 -> I4
```

- читаем поток инструкций по порядку
- но исполняем **готовые** инструкции раньше
- commit обычно сохраняем **в порядке программы**

4. Какие зависимости мешают переупорядочиванию

Тип	Расшифровка	Смысл
RAW	Read After Write	настоящая зависимость по данным
WAR	Write After Read	конфликт по имени регистра
WAW	Write After Write	конфликт по порядку записей

RAW нельзя ломать

WAR / WAW часто можно убрать через renaming

4. Какие зависимости мешают переупорядочиванию. Пример

Тип зависимости	Пример
RAW (Read After Write)	<pre>add x5, x1, x2 // записали x5 sub x6, x5, x3 // читаем новый x5</pre>
WAR (Write After Read)	<pre>add x7, x5, x6 // читаем старый x5 sub x5, x1, x2 // потом пишем новый x5</pre>
WAW (Write After Write)	<pre>add x5, x1, x2 // первая запись в x5 sub x5, x3, x4 // вторая запись в x5</pre>

4. Register renaming

Проблема:

- архитектурных регистров мало
- разные инструкции могут конфликтовать по имени

Идея:

Архитектурный x5

Старое значение x5 -> P12

Новое значение x5 -> P19

- внутри ядра больше физических регистров
- устраняются ложные конфликты WAR/WAW

4. Упрощённая схема ОоО-ядра

Fetch -> Decode -> Rename -> Schedule -> Execute -> Write Result -> Commit

Этап	Назначение
Rename	убрать ложные зависимости
Schedule	ждать готовности операндов
Execute	запускать готовые инструкции
Commit	фиксировать результат в нужном порядке

4. Reorder Buffer

Инструкция поступает в ROB



Результат вычислен



Ждёт когда выполнятся старые инструкции



Результат зафиксирован

ROB помогает:

- хранить временные результаты
- commit по порядку
- корректно отменять спекулятивный путь

4. Что выигрывает OoO

Сценарий	Польза OoO
долгий <code>load</code>	блокировка только зависимых инструкций
разная латентность операций	выполнить готовые по ресурсам инструкции
независимые инструкции	использовать параллелизм эффективнее

Цена: сложная микроархитектура и более тяжёлая верификация

5. Многопоточность

5. Идея multithreading

Если один поток застрял,
можно занять pipeline другим потоком

Thread A stalled



Core switches / issues Thread B

Многопоточность помогает скрывать:

- memory latency
- stalls
- ожидание данных

5. Coarse-grained multithreading

Такты:	1	2	3	4	5	6	7	8
Поток A:	A	A	A	A	stall			
Поток B:						B	B	B

- ядро некоторое время работает с одним потоком
- при большой задержке переключается на другой

Плюс: проще

Минус: часть времени всё равно теряется

5. Fine-grained multithreading

Такты:	1	2	3	4	5	6
Поток:	A	B	A	B	A	B

- переключение происходит очень часто
- можно почти каждый такт выбирать другой поток

Плюс: хорошо скрывает латентность

Минус: сложнее управление и хранение контекста

5. SMT — simultaneous multithreading

Одно ядро, несколько потоков одновременно

Такт N:

ALU0 <- Thread A

ALU1 <- Thread B

- разные ресурсы ядра могут использовать инструкции разных потоков
- цель: не оставлять функциональные блоки пустыми

Плюс: высокая загрузка ресурсов

Минус: аппаратная сложность ещё выше

5. Кто делит программу на потоки?

Важно различать **software threads** и **hardware threads**

Уровень	Кто решает
Software threads	программист / runtime / ОС
Hardware threads	микроархитектура ядра

То есть:

- программу на потоки обычно делит **ПО**
- сколько потоков ядро умеет держать одновременно — решает **железо**

5. Чем определяется количество потоков?

На уровне ПО

- сколько параллелизма есть в задаче
- накладные расходы на синхронизацию
- число доступных ядер / hardware threads

На уровне железа

- сколько аппаратных контекстов заложено в ядре
- сколько есть ресурсов
- насколько сложную микроархитектуру готовы поддерживать

6. Сравнение подходов

6. Сравнение методов

Подход	Влияет на	Основная идея
Branch prediction	control hazards	заранее угадать <code>PC</code>
Speculation	простой после branch	выполняем инструкции по предсказанному пути
OoO	ожидание данных / разная латентность	выполняем доступные по ресурсам инструкции раньше
Multithreading	stalls / memory latency	используем другой поток

6. Рост критериев вместе с производительностью

Больше производительность
↑↓
Больше сложность

Обычно растут:

- площадь логики
- число внутренних буферов
- энергопотребление
- сложность отладки
- сложность верификации

6. Choose Your Pain Level

1. Простой pipeline
2. Pipeline + branch prediction
3. Pipeline + speculation
4. Out-of-order execution
5. OoO + multithreading / SMT

Чем выше ступень:

- тем лучше использование параллелизма
- тем сложнее ядро

7. Почему следующим bottleneck становится память

7. Дальнейшие ограничения

Быстрое ядро <-----> Медленная память

Можно ускорять исполнение:

- prediction
- speculation
- OoO
- multithreading

Но если данные далеко, ядро всё равно начинает ждать **память**

7. Следующий слой сложности

После ускорения pipeline на первый план выходят вопросы:

- зачем нужен **cache**
- как виртуальный адрес превращается в физический
- как работает **защита памяти**
- что делать с **исключениями и прерываниями**

То есть: процессор надо рассматривать уже как часть системы

Следующая лекция

Память, трансляция адресов, защита и прерывания в RISC-V